Build a complete production-ready web application called **SyncAudio**.

1. APP CONCEPT

SyncAudio is a real-time multi-device synchronized audio playback web app.

The main use case is:

A group of friends is together somewhere, but they don't have a Bluetooth speaker. Everyone has their own phone/laptop. One person creates a SyncAudio session and starts an audio track. Other people join the same session using a Room ID or shareable link. The same audio should then play on all connected devices at approximately the same time through their individual speakers.

The application should focus on making multiple devices behave like a temporary synchronized speaker system.

The UI should feel modern, premium, smooth and fun — not like a basic developer demo.

---

2. LANDING PAGE

When the website opens, show a beautiful landing page for SyncAudio.

Hero section:

SyncAudio
"Turn every device into one synchronized speaker."

Subtext:

"Create a room, invite your friends, and play audio together — perfectly in sync."

Two primary buttons:

Create Session
Join Session

Also show a small explanation:

♪ One audio
📱 Multiple devices
⚡ Real-time synchronization

Add subtle animations and modern transitions.

Do NOT make the interface unnecessarily complicated.

---

3. CREATE SESSION

When the user clicks **Create Session**:

Create a unique room/session.

Generate a short readable Room ID such as:

SYNC-7K92

or

A7X9P2

The creator becomes the **Host**.

Show:

Your SyncAudio Room

Room ID:
`A7X9P2`

Buttons:

- Copy Room ID
- Share Session
- Show QR Code
- Start Session

Also display:

Connected Devices: 1

The host should be able to see participants joining in real time.

Example:

```
Connected Devices

👑 You — Host
📱 Rahul
📱 Aman
📱 Neha
```

The room should remain active while the session is running.

---

4. JOIN SESSION

When the user clicks **Join Session**:

Show a clean input:

Enter Room ID

Example:

`A7X9P2`

Button:

Join Session

Also provide:

Scan QR Code

If the user opens a shared session link directly, automatically take them to the corresponding room.

If the Room ID doesn't exist, show:

"Session not found. Please check the Room ID."

If the session has ended:

"This session has ended."

---

5. SESSION / PLAYER SCREEN

Once a user joins a room, show the main SyncAudio player.

The host should have full playback controls.

Participants should be able to see the current playback state.

Main player UI:

- Audio artwork / waveform area
- Song/file name
- Current time

- Total duration
- Progress bar
- Play
- Pause
- Previous
- Next
- Seek
- Volume

Host controls should be clearly identified.

Example:



---

6. AUDIO INPUT

For the first version, support local audio files.

The Host should have:

Upload Audio

Allow common formats such as:

- MP3
- WAV
- OGG
- M4A where browser support allows

After selecting an audio file:

- Show filename
- Show duration
- Show loading/progress state
- Make it available to participants in the session
- Prepare the same audio for synchronized playback

Do not permanently store huge audio files unnecessarily.

For the MVP, design the architecture so that audio can be temporarily associated with a session.

---

7. REAL-TIME SYNCHRONIZATION

This is the most important functionality.

Use WebSockets / Socket.IO or another reliable real-time communication mechanism.

When the Host presses:

PLAY

the server should broadcast a synchronized playback command.

Do NOT simply tell every device:

"play now"

because network latency differs between devices.

Instead, use a server-based timing mechanism.

Example:

```
Host sends:

PLAY
position: 42.50
startAt: serverTime + 1000ms
```

Every connected device calculates the required delay and starts playback at the specified server timestamp.

This should minimize differences between devices.

Synchronize at least:

- Play
- Pause
- Seek
- Current position
- Track change
- Host changes
- Session state

Periodically compare client playback position with the authoritative session position and gently correct drift.

Avoid constantly restarting audio.

Use small playback-rate corrections or controlled seeking when the difference becomes significant.

---

8. CLOCK / LATENCY SYNCHRONIZATION

Implement a simple client-server time synchronization mechanism.

Clients should estimate:

- server time
- network round-trip time
- local clock offset

Use that information to calculate when playback should begin.

The goal is to make devices start as close together as practically possible.

Display a small status indicator:

 Synced

or

 Syncing...

or

● Connection Lost

Do not claim "perfect millisecond synchronization" in the UI.

Use wording such as:

Synchronized playback

---

9. HOST CONTROLS

The creator/host should have control over playback.

Host can:

- Play
- Pause
- Seek
- Change track
- Add/remove audio
- End session

Participants should normally NOT be able to control global playback.

Clearly show:

HOST CONTROLS PLAYBACK

Participants can control their own:

- Device volume
- Mute/unmute

Changing one participant's volume must NOT change the volume of other devices.

---

10. PARTICIPANT MANAGEMENT

Show all connected devices.

Example:

```
People in this Session

👑 Gijendar — Host
📱 Device 2
📱 Device 3
📱 Device 4
```

Show connection status.

If a device disconnects, remove it from the active participant list.

If it reconnects, allow it to resynchronize with the current playback position.

---

11. ROOM SHARING

Make sharing extremely easy.

After creating a room, show:

Room ID

```
A7X9P2
```

Session Link

A shareable URL containing the room ID.

Buttons:

Copy Link

Share

Show QR

The QR code should encode the session URL.

A friend can scan it and directly reach the Join Session screen for that room.

---

12. PLAYLIST

Allow the host to create a temporary playlist.

Example:

```
QUEUE

01  Believer.mp3
02  Shape of You.mp3
03  Perfect.mp3
04  Night Changes.mp3
```

Host can:

- Add audio
- Remove audio
- Reorder audio
- Play next
- Play previous

When a track changes, all participants should automatically synchronize to the new track.

---

13. SESSION STATES

A session can have these states:

- Waiting
- Active
- Paused
- Ended

When created:

Waiting for friends...

When playback starts:

Playing

When paused:

Paused

When host ends it:

Session Ended

Participants should be redirected appropriately when the session ends.

---

14. RECONNECTING

If a participant temporarily loses internet connection:

Show:

Connection lost — reconnecting...

When connection returns:

Automatically reconnect to the room.

Fetch the current session state.

Synchronize the audio to the current playback position.

The user should NOT have to manually reload the page.

---

15. DATABASE

Use Supabase PostgreSQL if database persistence is required.

Suggested tables:

rooms

```
id
room_code
host_id
status
current_track
current_position
is_playing
created_at
updated_at
```

participants

```
id
room_id
display_name
session_id
joined_at
last_seen
is_connected
```

tracks

```
id
room_id
file_name
file_url
duration
position_in_queue
created_at
```

Do not store unnecessary playback events in the database.

Real-time playback state should primarily be handled through the WebSocket layer.

---

16. USER IDENTITY

Do NOT require complicated account registration for the MVP.

When joining a room, ask:

Enter your display name

Example:

```
Gijendar
```

Then show that name to other participants.

Generate a temporary session/device ID automatically.

The Host can optionally enter their name while creating the room.

---

17. RESPONSIVE DESIGN

The application must work beautifully on:

- Android phones
- iPhones
- Tablets
- Laptops
- Desktop browsers

Mobile experience is extremely important because the primary use case involves multiple phones.

Make buttons large enough for touch interaction.

Do not overcrowd the mobile player screen.

---

18. DESIGN STYLE

Use a premium modern music-app aesthetic.

Suggested design direction:

- Dark background
- Beautiful gradients
- Soft glowing accents
- Large typography
- Rounded cards
- Smooth transitions
- Subtle glass-like surfaces
- Modern icons
- Minimal but premium UI
- Music visualizer/waveform animation

Do NOT overuse glassmorphism.

Keep the interface clean and professional.

Suggested accent style:

Purple / blue / cyan gradient accents are acceptable.

Use animations sparingly and smoothly.

Add subtle animated background elements on the landing page.

---

19. RESPONSIVE PLAYER

On mobile, the player should look similar to:



Make it feel like a real music player.

---

20. CONNECTION / SYNC STATUS

Always give users useful feedback.

Examples:

 Connected
 Synchronized
 Synchronizing
● Disconnected

Also show:

4 devices connected

Do not expose technical WebSocket errors to normal users.

---

21. ERROR HANDLING

Handle:

- Invalid Room ID
- Room not found
- Room ended
- Host disconnected
- Audio upload failure
- Unsupported audio format
- Network disconnected
- WebSocket connection failure
- Audio playback blocked by browser autoplay policy
- Participant reconnect
- Empty playlist

For browser autoplay restrictions:

Show a clear button:

Tap to Start Audio

because some browsers require user interaction before audio can play.

---

22. HOST DISCONNECTION

If the Host disconnects temporarily:

Give the host a short grace period to reconnect.

If the host permanently leaves, either:

1. End the session

or

2. Transfer host privileges to another participant.

For the MVP, ending the session is acceptable, but implement the architecture so host migration can be added later.

---

23. SECURITY

Do not expose Supabase service-role keys in frontend code.

Use environment variables for secrets.

Validate Room IDs on the backend.

Do not trust client-side playback state blindly.

Validate that only the host can issue global playback commands.

Sanitize display names.

Limit room/participant abuse with reasonable rate limits.

---

24. PROJECT STRUCTURE

Use a clean maintainable structure.

Separate:

- frontend
- backend
- WebSocket logic
- database
- components
- utilities
- styles

Keep synchronization logic isolated in its own module so it can be improved later.

Use reusable components instead of putting everything into one huge file.

---

25. IMPORTANT MVP PRIORITY

Prioritize these features first:

1. Create Session
2. Generate Room ID
3. Join Session
4. Real-time participant list
5. Host audio upload
6. Audio playback
7. Play synchronization
8. Pause synchronization
9. Seek synchronization
10. Reconnection synchronization
11. Shareable room link
12. QR code
13. Responsive mobile UI

Only after these work properly, add:

- Playlist
- Advanced visualizer
- Host migration
- More advanced sync correction
- Session history

---

26. VERY IMPORTANT AUDIO ARCHITECTURE

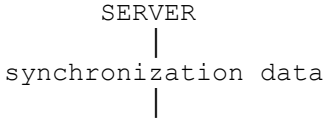
Do NOT continuously send the raw audio stream through the WebSocket server.

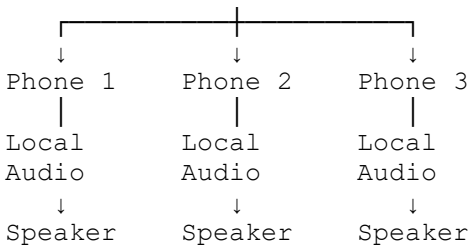
WebSocket should primarily transmit lightweight commands/state such as:

```
PLAY
PAUSE
SEEK
TRACK_CHANGED
SYNC
```

The actual audio should be loaded/played locally by each participant whenever possible.

The goal is:





This keeps bandwidth requirements much lower.

---

27. LANDING PAGE COPY

Use this exact style:

SyncAudio

One room. Every speaker. One beat.

"Turn your friends' phones into a synchronized speaker system."

Buttons:

Create Session

Join Session

Small text:

"No Bluetooth speaker? No problem."

---

28. ROOM PAGE COPY

Host:

Your session is ready ♪

Share the room with your friends

Room ID:

`A7X9P2`

Copy ID

Share Link

Show QR

Then:

Waiting for friends...

```
1 device connected
```

---

29. USER EXPERIENCE

The entire process should feel extremely simple:

```
Open SyncAudio
       ↓
Create Session
       ↓
Get Room ID / QR
       ↓
Friends Join
       ↓
Host selects audio
       ↓
Press Play
       ↓
♪ Everyone hears it together
```

No unnecessary registration.

No complicated setup.

No confusing settings.

---

30. FINAL REQUIREMENT

Build the actual working web application, not just a static UI mockup.

All major buttons must work.

Create/join rooms must work.

Real-time participant updates must work.

WebSocket synchronization must work.

Audio playback must work.

Play/pause/seek synchronization must work.

Room sharing must work.

QR joining must work.

The application should be structured so it can be deployed to production.

Use environment variables for configuration.

Provide a clear README with:

- Installation
- Environment variables
- Supabase setup
- Local development
- Production deployment
- WebSocket configuration

Before considering the project complete, test the core flow with multiple browser/device sessions:

Device A → Create Room → Device B/C → Join Room → Host uploads audio → Host presses Play → B/C synchronize → Host pauses → B/C pause → Host seeks → B/C seek.

The final result should feel like a polished real-world product called **SyncAudio**, not a basic coding demo.